CalWidget v2 Calibration Technical Documentation

EOVSAPY

April 14, 2026

Contents

1 Purpose and Scope **2**

2 High-Level Calibration Model **2**

3 Inputs and Internal Data Products **3**
3.1 Supported Inputs . . . 3
3.2 Reference Frames and Symbols . . . 4

4 Pre-Correction of Fine-Channel Visibilities **4**
4.1 Feed Rotation and Parallactic-Angle Handling . . . 4
4.2 Time-Drift Correction . . . 4

5 Uniform In-Band Delay Solution **4**
5.1 Per-Band Linear Phase Fit . . . 4
5.2 Notebook-Matched Weighted Mean . . . 5
5.3 Applied Delay Convention . . . 5

6 Delay-Solution State **5**

7 Relative Phase Model **6**
7.1 Reference Convention . . . 6
7.2 Joint X/Y Constraint . . . 6
7.3 Antenna-Dependent Model Class . . . 6
7.4 Manual Relative Correction . . . 6

8 Residual Diagnostics **7**
8.1 Per-Band Residual Phase . . . 7
8.2 Residual Delay Per Band . . . 7
8.3 Residual-Specific Masking . . . 7
8.4 Apply Residual Fit . . . 7

9 Automatic and Manual Antenna Flagging **8**
9.1 Y-X Residual RMS . . . 8
9.2 Flag Layers . . . 8

10 Time Masking **8**

11 Scan Classification and Feed Separation **8**

12 Persistence **9**
12.1 Refcal Sidecar 9
12.2 Daily Model NPZ Export 9

13 Frontend Payload Architecture **9**

14 Recommended Operator Workflow **13**
14.1 High Feed 13
14.2 Low Feed 13

15 Known Boundaries **13**

16 File Map **14**

17 Summary **14**

1 Purpose and Scope

This note documents the calibration logic implemented in the browser-native CalWidget v2 path inside `eovsapy.calwidget_html`. The goal is to give an operator- and developer-facing description of what the code solves, what state it stores, how interactive edits affect the solution, and what products are exported to SQL or NPZ.

The document covers:

- fine-channel reference calibration analysis for refcals,
- in-band delay fitting,
- relative phase modeling and residual diagnostics,
- time masking and band masking,
- manual and automatic flagging layers,
- exported sidecar and daily NPZ products,
- the intended operational workflow for HI and LO feed calibration.

This note does not attempt to replace the scientific notebooks. It describes the current production-oriented implementation in the v2 widget.

2 High-Level Calibration Model

The v2 path treats a selected refcal as the anchor dataset for a given feed and solves the calibration in layers:

1. read and pre-correct fine-channel visibilities,
2. fit one *uniform in-band delay* per antenna and polarization,

3. apply that delay to all fine channels,
4. build a *relative phase model* referenced to Antenna 1,
5. diagnose remaining per-band residual phase and per-band residual delay,
6. optionally apply a *residual in-band correction*,
7. export the tuned model and flags.

The design intentionally separates these layers because they solve different problems:

- the in-band delay solves the channel-scale phase slope within each band,
- the relative phase model solves the multi-band phase trend relative to the reference antenna,
- the residual diagnostics solve what remains after the previous two corrections and are used as a suggest-only or apply-on-click refinement path.

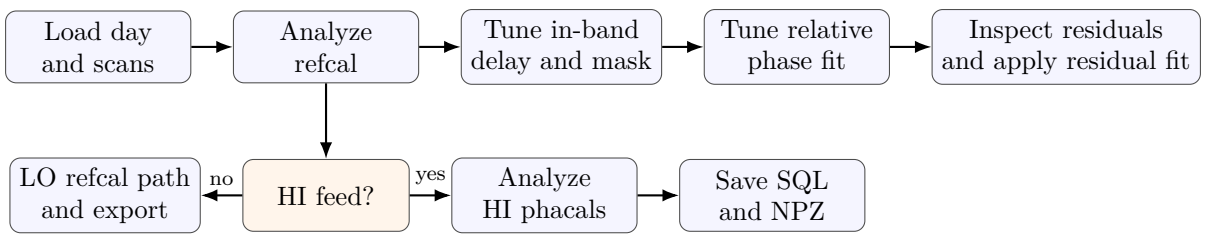

Figure 1: Operator workflow block diagram for CalWidget v2. The HI path includes in-band, relative-phase, and residual refinement before phacal analysis. The LO path remains separate.

3 Inputs and Internal Data Products

3.1 Supported Inputs

The refcal/phacal preparation path can ingest either:

- raw IDB/UDB visibility data, or
- an NPZ file written by the legacy notebook/analysis path.

The imported scan is converted into a `ScanAnalysis` object that stores:

- time stamps,
- source and scan metadata,
- layout information,
- fine-channel visibility cubes,
- corrected fine-channel visibility cubes,
- band-averaged products,
- time masks and flag products,
- the active delay solution and derived model state.

3.2 Reference Frames and Symbols

Throughout this note:

- a is the antenna index,
- $p \in \{X, Y\}$ is polarization,
- b is the band index,
- f is frequency in GHz unless stated otherwise,
- $\tau_{a,p}$ is a delay in ns,
- ϕ is wrapped phase in radians.

4 Pre-Correction of Fine-Channel Visibilities

4.1 Feed Rotation and Parallactic-Angle Handling

The v2 fine-channel path follows the same date-dependent behavior as the legacy Tk widget:

- before 2025-07-15, full X/Y feed-rotation and parallactic-angle unrotation are applied,
- from 2025-08-08 onward, only the Ant 4 parallactic-angle wrap fix is applied,
- between these dates the code follows the same cutoff logic as the legacy path.

This is important because applying the old full feed-rotation correction to later data changes the fine-channel phase pattern and breaks consistency with legacy calibration products.

4.2 Time-Drift Correction

Before fitting in-band delays, the code applies a legacy-style time-drift correction in the fine-channel domain. For each antenna and polarization:

1. a per-band time slope is fit from the band-averaged phase versus time,
2. only bands with phase-fit scatter below a quality cutoff are kept,
3. a median drift coefficient is formed,
4. all fine channels in a band are rotated by the corresponding time drift.

This makes the fine-channel fitting path more consistent with the legacy band workflow and reduces slow time-dependent phase drift before delay solving.

5 Uniform In-Band Delay Solution

5.1 Per-Band Linear Phase Fit

For each antenna a, polarization p, and fitted band b, the code computes a linear phase fit

$$\phi_{a,p,b}(f) \approx \phi_{0,a,p,b} + 2\pi f \tau_{a,p,b}. \tag{1}$$

The per-band solve stores:

- `per_band_delay_ns`,
- `per_band_phase0`,
- `per_band_std`.

5.2 Notebook-Matched Weighted Mean

The fitted uniform in-band delay is the weighted mean over valid per-band delays:

$$\tau_{a,p}^{\mathrm{fit}} = \frac{\sum_b w_{a,p,b}\tau_{a,p,b}}{\sum_b w_{a,p,b}}, \qquad w_{a,p,b} = \frac{1}{\sigma_{a,p,b}^2}. \tag{2}$$

The important implementation choice is that $\sigma_{a,p,b}$ is kept as the raw phase-fit scatter returned by `lin_phase_fit`, in radians. It is *not* converted into a delay-domain uncertainty. This matches the notebook reference path and avoids over-weighting wider high-frequency bands.

5.3 Applied Delay Convention

The active delay is applied to all channels in a band using the band midpoint:

$$V'_{a,p}(f) = V_{a,p}(f)\,\exp\left[-i\,2\pi\tau_{a,p}^{\mathrm{active}}\,(f - f_{\mathrm{mid},b})\right]. \tag{3}$$

This removes the fitted in-band slope without forcing an arbitrary phase shift at the band center.

6 Delay-Solution State

The active refcal stores the in-band and relative-fit state in a `DelaySolution` object. The main fields are:

- `fitted_ns`: notebook-matched fitted in-band delay,
- `active_ns`: currently applied in-band delay,
- `relative_ns`: display-only multiband relative correction,
- `kept_band_mask`: main in-band mask for the in-band delay solve,
- `residual_kept_band_mask`: residual-panel-specific mask,
- `residual_auto_band_mask`: residual auto mask,
- `manual_ant_flag_override`: operator whole-antenna flag layer,
- `manual_ant_keep_override`: operator whole-antenna keep layer.

The main kept-band mask and the residual kept-band mask are intentionally separate. The former controls the main in-band delay solution. The latter controls only the residual in-band refinement path.

7 Relative Phase Model

7.1 Reference Convention

The relative phase panel uses Antenna 1 as the reference.

- For Antenna 1, the panel displays the corrected absolute phase.
- For antennas $a > 1$, the panel displays the corrected phase of Antenna a relative to Antenna 1.

7.2 Joint X/Y Constraint

The model uses both X and Y polarization together. The code assumes that $Y - X$ is approximately constant for a given antenna after delay correction, so the X and Y phase tracks are not fit independently. The procedure is:

1. compute delay-corrected X and Y phases,
2. estimate a circular mean $Y - X$ offset,
3. align the X and Y tracks to one shared phase branch,
4. fit a shared phase model,
5. reconstruct the Y model by re-applying the fitted $Y - X$ offset.

7.3 Antenna-Dependent Model Class

The code uses different model classes for the reference and non-reference antennas:

- **Antenna 1**: a weighted Chebyshev polynomial model on the coarse band-sampled phase trend,
- **Antennas 2+**: a weighted linear model in frequency.

Low frequency is up-weighted by a factor proportional to $1/\sqrt{f}$ to reduce the leverage of noisy high-frequency channels.

7.4 Manual Relative Correction

The `Relative Phase Delay Editor` applies a display-only additional delay on top of the automatically fit multiband baseline. For non-reference antennas the slope changes, then the intercept is re-fit from the time-averaged fine-channel data so the line is anchored back to the data after the slope edit.

This path is separate from the in-band delay editor:

- **Anchor Refcal Delay Editor**: changes the applied in-band delay `active_ns`,
- **Relative Phase Delay Editor**: changes the displayed multiband relative-fit overlay only.

8 Residual Diagnostics

8.1 Per-Band Residual Phase

After the relative model is formed, the code computes residual phase by subtracting the model from the corrected phase and re-wrapping the difference to $[-\pi, \pi]$.

The residual panel shows two different fit layers:

- **Multiband Fit** in red: the residual-guided multiband suggestion path,
- **Inband Fit** in blue: an in-band style linear fit computed separately within each band from the residual phase.

8.2 Residual Delay Per Band

The residual delay panel computes, for each band, a post-correction scalar delay estimate from the corrected complex visibility. It also computes an all-band residual delay estimate. These are diagnostics; they are not the primary fit.

8.3 Residual-Specific Masking

The residual panel now has its own mask state, independent of the main in-band kept-band mask. The purpose is to exclude obviously bad bands from the residual in-band refinement without altering the main in-band fit window.

The residual auto mask uses two gates:

1. residual phase scatter gate:
$$\sigma^{\mathrm{res}}_{a,p,b} \leq \sigma_{\mathrm{thr}}, \tag{4}$$
where σ_{thr} is a user-adjustable residual scatter threshold, defaulting to 1.0 rad,
2. residual delay robust outlier gate:
$$\left|\tau^{\mathrm{res}}_{a,p,b} - \mathrm{median}(\tau^{\mathrm{res}})\right| \leq 3\,\mathrm{MAD}(\tau^{\mathrm{res}}). \tag{5}$$

If too few bands are finite or the MAD is zero, the delay gate is disabled and only the scatter gate is used.

8.4 Apply Residual Fit

When the operator clicks `Apply Residual Fit`, the code:

1. commits any staged residual masks,
2. computes one residual in-band correction per antenna and polarization from the committed residual mask,
3. adds that correction to `active_ns` for all antennas and both polarizations,
4. rebuilds corrected diagnostics and dependent products.

This path does *not* change:

- the main kept-band mask used by the original in-band solve,
- the multiband relative baseline fit state.

9 Automatic and Manual Antenna Flagging

9.1 Y-X Residual RMS

For each antenna, the code computes a $Y - X$ residual RMS from the difference between the observed $Y - X$ phase and the fitted constant $Y - X$ model. This is a quality metric for phase consistency between the two polarizations.

The threshold is operator-adjustable in the `Relative Phase + Fit` section. The current default is 1.5 rad.

9.2 Flag Layers

Three layers of flag semantics are kept distinct:

- `legacy_flag`: legacy/SQL-style flag array,
- `v2_flag`: current tuned v2 flag state,
- `model_flag`: export flag state for the fitted model path.

In the interactive relative-phase panel, whole-antenna manual keep/flag overrides are stored separately from the automatic threshold decision. This allows the operator to bring back an automatically flagged antenna or exclude a nominally acceptable one.

10 Time Masking

Time masking is handled in the browser-native time-history path. The main ideas are:

- intervals are defined in Julian Date,
- multiple antenna-band targets can share one interval group,
- staged time masks are queued until the operator clicks `Apply Mask`,
- the heatmap distinguishes pending and committed masked cells.

The fine-channel diagnostic plots use time-averaged visibilities after applying the active browser time-mask state. Therefore a time-mask update changes the data shown in the in-band and residual panels.

11 Scan Classification and Feed Separation

HI and LO feed scans are handled separately. The code attempts to classify feed kind from scan metadata such as project and FSeq information, with duration used only as a fallback when explicit metadata is unavailable.

Operationally:

- HI and LO refcals should be solved separately,
- HI phacals use the tuned HI reference model,
- LO calibration usually does not require the same relative multiband phase refinement depth as HI.

12 Persistence

12.1 Refcal Sidecar

Each tuned refcal can write a sidecar NPZ under `/common/webplots/phasecal`. The sidecar stores the fitted and active delay solution, mask state, manual flag overrides, and scan metadata. The sidecar is used to restore the calibration state when returning to the same refcal.

12.2 Daily Model NPZ Export

The daily bundle export writes one NPZ archive containing one entry per analyzed scan. For each exported scan, the bundle stores:

- fine-channel frequency grid,
- band frequency grid,
- fitted model phase on the fine grid,
- fitted model phase on the band grid,
- `legacy_flag`,
- `v2_flag`,
- `model_flag`,
- manual antenna keep/flag overrides,
- $Y - X$ residual RMS and threshold,
- scan metadata.

Flags are saved as explicit arrays. The phase arrays are *not* encoded by blanking or NaN-masking to indicate flags.

13 Frontend Payload Architecture

The browser uses three top-level data channels:

- heatmap payload,
- time-history payload,
- overview payload.

Within the overview payload, each plot section has its own payload builder. The server can return:

- a full overview object, or
- `overview_updates` containing only the sections affected by a given action.

This split is important for performance:

- initial load fetches all top-level payloads,
- many button actions return only the relevant patched sections,
- sparse per-antenna edits can update only one antenna's panels.

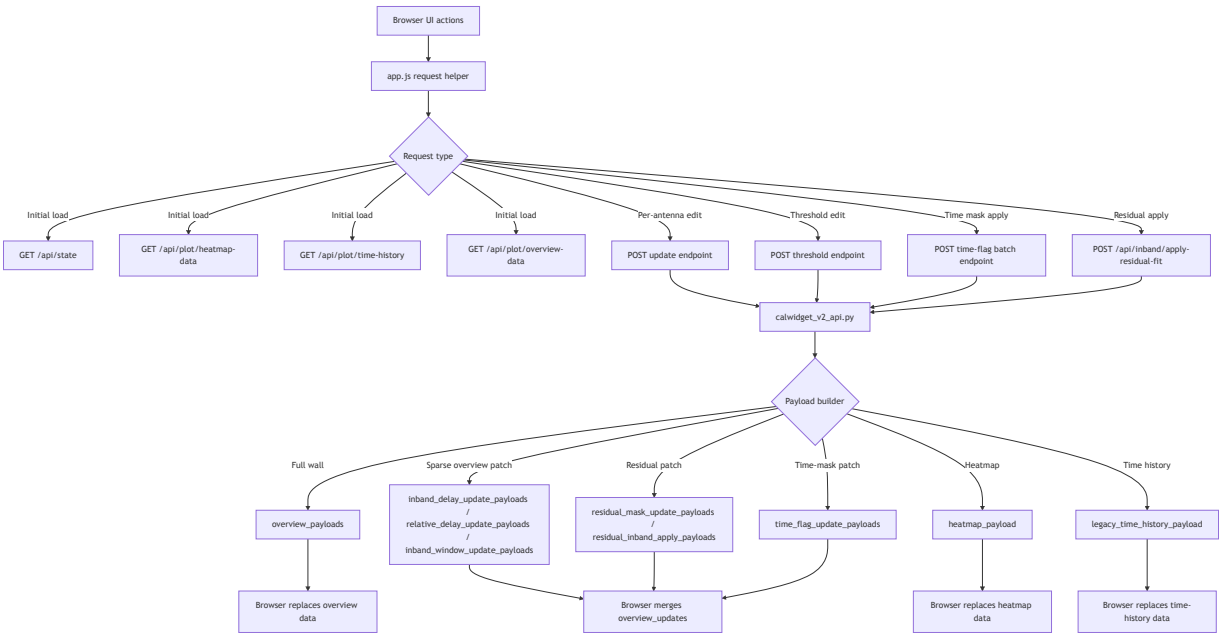

Figure 2: API payload flow. Initial page load uses separate top-level fetches, while most interactive actions return scoped payload patches.

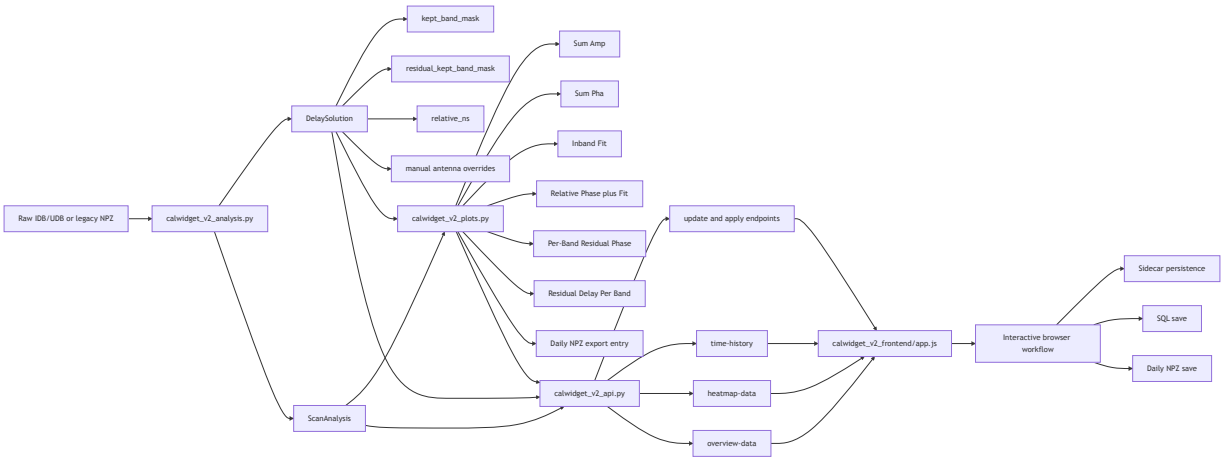

Figure 3: Code architecture for the v2 calibration path. Analysis state, payload generation, API routing, frontend interaction, and export are separated into distinct modules.

14 Recommended Operator Workflow

14.1 High Feed

The intended HI workflow is:

1. select and analyze one HI refcal,
2. tune the in-band delay mask and solve,
3. inspect `Relative Phase + Fit`,
4. apply multiband relative suggestions only when needed,
5. inspect `Per-Band Residual Phase`,
6. tune residual masks and apply residual fit,
7. verify flags, $Y - X$ residual RMS, and whole-antenna keep/flag decisions,
8. analyze the HI phacals,
9. save SQL products as needed,
10. save the daily NPZ bundle.

If both morning and evening HI refcals exist, the operator may:

- use only one anchor, or
- choose one canonical anchor and keep the second as secondary drift metadata.

14.2 Low Feed

The LO workflow is separate:

1. analyze the LO refcal,
2. verify amplitude and phase consistency,
3. save LO products,
4. export to the daily NPZ bundle if desired.

15 Known Boundaries

The current implementation has several explicit boundaries:

- the multiband and residual correction paths are separate by design,
- residual masking affects residual in-band refinement only,
- the Chebyshev Antenna 1 model is an analytic approximation, not a full physical instrument model,
- the daily NPZ export writes whatever products already exist in the current session; it does not synthesize missing analyses.

16 File Map

The main implementation files are:

- `eovsapy/calwidget_html/calwidget_v2_analysis.py` for scan preparation, delay-solution state, masking state, and sidecar persistence,
- `eovsapy/calwidget_html/calwidget_v2_plots.py` for diagnostic model generation and export payloads,
- `eovsapy/calwidget_html/calwidget_v2_api.py` for the HTTP API,
- `eovsapy/calwidget_html/calwidget_v2_frontend/app.js` for the browser interaction model.

17 Summary

CalWidget v2 is not just a re-skin of the legacy widget. It is a layered fine-channel calibration system with explicit state for:

- notebook-matched in-band delay fitting,
- joint X/Y multiband relative phase modeling,
- residual-specific bad-band masking,
- manual and automatic antenna flagging,
- sidecar persistence and daily model export.

That separation is the main design principle of the v2 path. It keeps the operator in control while still allowing partial automation and reproducible exports.